班　级　2203013

学　号　22009201086

西安電子科技大學

本科毕业设计论文

题　　目　轻量化密态区块链数据库可验证查询方法设计与实现

学　　院　计算机科学与技术学院

专　　业　计算机科学与技术

学生姓名　杨芸茜

导师姓名　王运帷

摘 要

本研究论文深入分析了牛马动力学在土木工程中的应用，特别是在打灰技术中的作用，以及这一传统实践如何与现代工程实践相结合。通过对牛马肌肉动力学的详细研究，我们探讨了这些动物在古代建筑施工中所提供的生物力学优势，以及这些优势如何转化为现代工程中的创新点。例如，牛马在搬运重物时展现出的高效率和低能耗特性，为现代起重机械设计提供了生物模拟的灵感。

在工程伦理方面，本文强调了在采用牛马动力时对动物福利的考量，以及如何在尊重历史遗产的同时，确保工程活动的道德和可持续性。我们通过实验数据展示了在特定条件下，牛马动力与传统机械动力在施工效率和成本效益上的差异。例如，实验表明，在某些地形复杂的建筑现场，牛马动力的使用可以减少约 30

此外，本文还探讨了牛马动力在现代土木工程中的应用潜力，特别是在偏远地区或基础设施不发达的地区。通过对比分析，我们发现在这些地区，牛马动力不仅是一种经济有效的选择，而且还能提供对环境影响较小的施工方案。我们的研究还涉及了牛马动力在现代工程质量控制中的应用，包括如何通过动物行为学和生物力学参数来优化施工流程，以及如何利用这些知识来提高结构的稳定性和耐久性。

关键词：牛马动力学　　工程创新　　工程伦理　　施工方法　　工程质量控制

ABSTRACT

This study examines the integration of cattle and horse power into modern civil engineering, drawing on their historical use in plastering techniques. It highlights the biomechanical advantages of these animals, such as efficiency and low energy consumption, and their potential to inspire modern crane design. The paper also addresses engineering ethics, emphasizing animal welfare and the sustainability of using animal power in construction. Experimental findings show that in complex terrains, animal power can significantly reduce construction time and costs while minimizing environmental impact. The research further explores the application of animal dynamics in remote areas with limited infrastructure, where it offers economic and environmental benefits. The study also considers how animal behavior and biomechanics can optimize construction processes and enhance structural stability.

Keywords: Animal Dynamics Engineering Innovation Engineering Ethics Construction Methods Engineering Quality Control

目　录

第一章　绪论

1.1　选题背景和意义

随着云计算技术的不断发展，越来越多的用户选择将数据加密外包到云服务商，既可以节省本地存储空间，又能随时随地访问数据。但是加密会破坏数据的多种语义关系，致使传统搜索技术失效。于是 Song 等人于 2000 年提出可搜索加密技术，实现了在不泄露文件信息到的前提下对密文进行检索，有效保护用户隐私。可搜索加密分为堆成可搜索加密和公钥可搜索加密。对称可搜索加密效率更高，因而得到更广泛的研究。现有大多数 SSE 方案中，服务器的安全模型定义为诚实且好奇，即服务器会正确地存储数据，诚实地执行搜索算法，并返回正确地结果。但是，在公有云存储场景下，云服务器可能是不诚实的，会为了节省计算资源而向用户返回不完整不正确的搜索结果。因此，对密文检索到的结果进行验证十分必要。

区块链的引入为解决传统可搜索加密中存在的问题提供了解决方案。区块链是一种区块数量单调增长的数据结构，它分布式存储在网络中的对等节点之间。尽管网络中的对等方可能不会相互信任，但是区块链从两个方面确保了数据完整性。首先，借助哈希链技术，存储在区块链上的数据是不可变恶的。其次，根据共识协议，区块链可确保所有对等方都维护相同的数据副本。区块链所能提供的存储空间、计算能力和其它网络资源往往有限且非常昂贵，现有的基于区块链的可搜索加密方案复杂度高，难以实际部署，并且可验证数据集大，对边缘设备计算量要求高。因此，如何设计基于区块链高效的、可验证的密文检索方案成为新的挑战。针对以上问题，本文提出一种可验证查询方案，使得可验证效率提升。

1.2　国内外研究现状

可搜索加密技术允许用户在加密数据上进行检索，是实现隐私保护和数据可用平衡的关键。自 Song 等人首次提出对称可搜索加密方案以来，该领域已经取得了长足的发展。其演进方向主要沿着基础方案优化、可验证性增强以及与新兴技术融合三个方向展开。

早期可搜索加密方案主要关注在如何保证隐私的前提下实现高效数据检索。Song 等人的开创性工作实现了密文上检索，但是需要线性扫描整个数据集，进行逐文档逐关键词检索，效率低下。Goh 提出基于陷门函数的可搜索加密方案，同时使用 Bloom Filter 技术，使得检索复杂数与数据库文档数有关。Cutmola 引入倒排索引，检索复杂度为亚线性，即只与检索关键词包含的文档书有关，进一步提高检索效率。

初始的单关键词搜索返回大量满足检索条件的文档，很难满足用户精细化需求，Golle 等人首次提出两个非直接的连接关键词检索方案，但方案有大量模指数运算和双线对运算。Cash 等人提出首个亚线性的连接关键词查询方案 (Obvious Cross-Tags,OXT)，并且可扩展为支持关键词的布尔查询。Lai 等人指出该方案会泄露结果模式。具体来说，就是 OXT 协议执行中，当客户端向服务器发送搜索令牌后，服务器能够确定地知道哪些加密文档 ID 与本次查询的布尔条件相匹配。Sun 将 OXT 扩展为多用户场景，提出了无交互的细粒度多用户 SSE 方案。Wang 指出该方案有验证的通信和计算代价，顺势提出了服务器端匹配的匿名属性 ABE 技术，在不泄露用户属性的情况下，服务器可以判断用户能否解密密文。用此技术加密文档标识，服务器可以仅仅返回满足检索条件且用户能够解密的结果，降低了通信代价。Kamara 等人优化 OXT 在执行析取关键字检索（返回包干任意查询关键词的文档）时效率低下问题。

需要指出的是，上述方案均考虑诚实且好奇的服务器模型，即服务器会严格按照协议执行操作，并不会主动丢弃数据、返回错误结果或中途破坏协议，但是会被动记录和分析经受数据，从中挖掘用户隐私。然而，在实际由于服务器软硬件运行故障或节省网络带宽和计算资源，它可能返回给用户不正确或不完整的检索结果。因此，对检索结果进行验证时有必要的。对于单关键词检索 SSE 方案来讲，使用布隆过滤器、哈希函数、聚合器、默克哈希树、哈希链等验证工具可以实现。多模式的检索验证是比较有挑战性的。Azraoui 等人方案利用 Cuckoo 哈希函数和聚合器，Sun 等人利用聚合器和默克尔哈希树实现连接关键词的公开可验证。然而，这两种验证方式均需要服务器检索出每个关键词的结果，然后求交集，会导致检索效率低和泄露多的问题。针对此，Wang 基于 OXT 的连接关键词检索方法，采用聚合器和采样检测技术提出了公开可验证的连接关键词检索方案。为支持的动态更新下的验证，Sun 等人基于双线性映射累加器及累加器树构建了支持

动态连接词查询的可验证方案；Zhu 等人提出动态模糊关键词可验证方案；Liu 等人则支持排序结果的验证。这些动态方案多依赖 RSA 累加器等公钥密码原语，验证效率有待提升。Zhang 等人基于多集哈希函数，Ge 等人基于对称加密分别构建高效的动态可验证方案，显著提升效率。然而，上述方案均依赖客户端本地验证，当用户本身不诚实时，可能会伪造验证结果，无法实现公平性。

区块链的区中心特性、完整性和不变性为构建高效可验证，公平的可搜索加密方案提供了解决方案。在基于区块链的方法中，区块链节点使用共识机制自动验证交易的合法性和正确性，任何数据都将被永久记录。Xu 等人首次系统性研究区块链数据库上可验证的布尔查询问题，提出额 vChain 的可验证查询框架，将能够对复杂多谓词聚合批量验证。Fang 等人提出一种多重索引结构来实现可验证查询，核心结构是 MB-tree，不适合跨块聚合数据和批量验证，验证数据集庞大。Zhang 等人利用变色龙向量承诺将本体与认证结构分离存储，区块链仅维护紧凑的、全局一致的根承诺。李宇溪等人将提出基于区块链索引的可验证布尔搜索加密，在本地设备计算资源消耗小，但在链上存在较多冗余信息。Guan 提出针对加密数据的分布式安全搜索方案，以解决电子商务场景中的数据安全问题。该方案引入联盟链和 PoA 机制，牺牲部分去中心化来提升效率。Zhang 等人提出的 BA-RMKABSE 满足多关键字特征，并使用向量空间模型计算查询关键字与文件之间的相关性，并基于分数排名，返回最符合用户需求的文件，从而节省带宽。

1.3　研究内容

在探讨牛马动力学在土木工程中的应用之前，有必要回顾一下相关领域的研究工作。

1.4　论文结构安排

安排论文结构

1.5　本章小结

本章小结

第二章　预备知识

本章基于在密文上的可验证查询框架，梳理出了在方案设计和具体实现中所用到的关键技术。

2.1　哈希函数与伪随机函数

2.1.1　抗碰撞哈希函数

抗碰撞哈希函数几乎是所有信息安全应用的必需组件，通常运用在密码存储和数据完整性检测等方面，因而也是本文构建可验证数据结构的基础。它能将任意长度的输入转换为具有固定长度的压缩数值，令 $H : \{0,1\}^* \to \{0,1\}^l$ 为一个哈希函数，它具备以下特性：

1. **单向性：**对于任何概率多项式时间的对手 $\mathscr{A}$，给定哈希值 z，找到输入 x 使得 $H(x) = z$ 的概率是可忽略的。该性质防止了攻击者从哈希值逆向恢复原始消息。

2. **抗碰撞性：**对于任何概率多项式时间的对手 $\mathscr{A}$，找到两个不同的输入 x 和 y，使得 $H(x) = H(y)$ 的概率是可忽略的。

在本文中，我们采用 BLAKE2b 算法。为实现复杂对象的哈希映射，我们定义可摘要特征，以确保系统内任何对象都能被确定性地序列化并映射为 32 字节的摘要，从而支持 Merkle 型结构的完整性验证。

2.1.2　伪随机函数

伪随机函数用于在不泄露原始信息的情况下生成确定性的标识符。令 $F : \mathscr{K} \times \mathscr{X} \to \mathscr{Y}$ 为一个伪随机函数族，其满足伪随机性：对于任何概率多项式时间的对手，在不知道密钥 k 的情况下，$F(k,\cdot)$ 的输出与从 $\mathscr{Y}$ 中均匀随机选取的函数在计算上不可区分。

在本文中，伪随机函数用于生成搜索令牌。具体地，我们采用带密钥的哈希构造，将主密钥 mk 与标准化后的关键字进行拼接，再经哈希函数 H 压缩得到令牌：$Token = H(mk \,\|\, kw)$。该方式既保证了相同关键字产生相同令牌，又避免了

直接暴露关键字明文。

2.2 密码学累加器

密码学累加器是本文实现可验证集合（交、并、差）的代数核心。

2.2.1 双线性映射基础

本文采用的累加器方案基于双线性群构造。令 $\mathbb{G}_1, \mathbb{G}_2$ 为阶为大素数 p 的循环加法群，$\mathbb{G}_T$ 为同阶的循环乘法群。若映射 $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ 满足以下属性，则称 e 为一个双线性映射：

双线性：对任意 $P \in \mathbb{G}_1, Q \in \mathbb{G}_2, a, b \in \mathbb{Z}_p$，有 $e(aP, bQ) = e(P, Q)^{ab}$。

非退化性：存在 $P \in \mathbb{G}_1, Q \in \mathbb{G}_2$，使得 $e(P, Q) \neq 1$。

可计算性：对任意 $P \in \mathbb{G}_1, Q \in \mathbb{G}_2$，$e(P, Q)$ 可在多项式时间内有效计算。

在本实验中，我们使用了 BN254 椭圆曲线，该曲线在保证 128 位安全强度的同时，提供了高效的配对运算性能。

2.2.2 累加器算法

// 这里 AI 率很高本文实现了一种支持多种可验证集合操作（如交∩、并∪、差∖）的高效累加器。其核心思想是将集合元素映射为多项式的根，并在群指数上进行运算。对于集合 $X = \{x_1, x_2, \ldots, x_n\}$，我们定义其特征多项式为：

$$P_X(z) = \prod_{x_i \in X} (z + x_i) \tag{2-1}$$

该累加器由以下五个算法组成：

$KeyGen(1^\lambda, U) \to pk$：输入安全参数 λ 和全集 U 的容量。随机选取秘密值 $s \in \mathbb{Z}_p^*$。输出公钥 $pk = (g, g^s, g^{s^2}, \ldots, g^{s^U}, h, h^s)$，。

$SetUp(X, pk) \to acc(X)$：输入集合 X 和公钥 pk。计算集合的累加值 $acc(X) = g^{P_X(s)} = g^{\prod_{x_i \in X}(s+x_i)}$，该值是一个常数大小的群元素，在 BN254 下为 32 字节，代表了整个集合的状态。

$Update(acc(X), acc(\Delta), pk) \to acc(X \pm \Delta)$：累加器支持增量更新。输入原累加值、增量集合 Δ 的累加值及公钥。插入为 $acc(X \cup \Delta) = acc(X)^{P_\Delta(s)}$，删除为 $acc(X \setminus \Delta) = acc(X)^{1/P_\Delta(s)}$。更新后的累加器基于新集合 $X \pm \Delta$ 计算，且无需访问

原集合的原始元素。

$Prove(X_1, X_2, opt, pk) \rightarrow (R, \pi_{opt})$：输入两个集合 X_1, X_2 和操作符 $opt \in \{\cap, \cup, \setminus\}$。算法首先计算结果集 $R = opt(X_1, X_2)$。随后，基于多项式恒等式生成证明 π_{opt}。例如，证明交集时，需计算满足 $P_{X_1} \cdot Q_1 + P_{X_2} \cdot Q_2 = P_R$ 的多项式 Q_1, Q_2，并将它们的群映射作为证明 π_{opt} 的组成部分。

$Verify(acc(X_1), acc(X_2), opt, \pi_{opt}, acc(R), pk) \rightarrow \{0, 1\}$：验证者利用双线性配对检查运算结果 R 是否正确。验证等式利用配对的乘法特性来验证指数上的多项式关系。如果 $R = opt(X_1, X_2)$，则返回 1，否则返回 0。

2.2.3　安全性假设

本方案的安全性建立在以下两个经典的困难性假设之上：

q-Strong-Diffie-Hellman (q-SDH)：假设令 $pub = (p, \mathbb{G}_1, \mathbb{G}_2, e, g, h)$ 为双线性映射参数。对于所有的多项式 q 和所有 PPT 的对手 $\mathscr{A}$，下述实验成功的概率可忽略不计：

$$\Pr\left[s \leftarrow \mathbb{Z}_p^*; \sigma = \{pub, g, g^s, \ldots, g^{s^q}, h, h^s\}; (c, w) \leftarrow \mathscr{A}(\sigma) : w = g^{\frac{1}{c+s}}\right] \approx 0 \quad (2\text{-}2)$$

该假设保证了对手无法在不知道 s 的情况下，为一个不在集合中的元素 c 伪造有效的成员证明。

q-Diffie-Hellman 指数 (q-DHE)：假设令 $pub = (p, \mathbb{G}_1, g)$ 为上述描述的双线性映射参数。它强调对于所有的多项式 q 和所有 PPT 的对手 $\mathscr{A}$，下述实验成功的概率可忽略不计：

$$\Pr\left[s \leftarrow \mathbb{Z}_p^*; \sigma = \{pub, g, g^s, \ldots, g^{s^{q-1}}, g^{s^{q+1}}, \ldots, g^{s^{2q-2}}\}; y \leftarrow \mathscr{A}(\sigma) : y = g^{s^q}\right] \approx 0 \quad (2\text{-}3)$$

该假设确保了即使公开了 s 的多项式幂次，特定的高次幂项依然在计算上不可获得。

2.3　Merkle 树

默克尔树是一种基于哈希函数构建的分层校验数据结构，旨在实现大规模数据集完整性的高效验证。其构造过程采用自底向上的哈希级联方式：首先将原始数据块 L_i 映射为叶子节点的哈希值；随后，每两个相邻子节点通过字符串连接并

再次执行哈希运算，生成其父节点 $N_{parent} = H(N_{left} \parallel N_{right})$，直至汇聚为唯一的顶层节点——默克尔根（Merkle Root）。默克尔根是对所有底层数据的密码学承诺，由于哈希函数的雪崩效应，数据集中任何微小的变动都会导致根哈希发生剧烈变化。基于此构造，默克尔树能够生成轻量级的成员身份证明（Membership Proof）。为了证实特定数据 L_1 存在于集合中，证明者无需传输完整数据集，仅需提供一条由该节点至根节点路径上所有必要的“兄弟节点”哈希值组成的默克尔路径。验证者（如区块链中的轻节点）利用目标数据与路径中的哈希值逐层向上还原计算，若最终结果与预存的根哈希一致，则证明该数据块的合法性。这种机制将验证复杂度从线性 $O(n)$ 降低至对数级 $O(\log n)$，极大地优化了分布式系统中的通信开销与验证效率。

以下图**??**为例，若验证者（如区块链轻节点）需验证数据块 64 (9Dog) 的存在性及其完整性，需持有已知的 Merkle 根哈希 (6c0a)，并从全节点获取目标数据块 64 及其对应的 Merkle 路径。在该示例中，路径由图中蓝色标记的兄弟节点哈希集合组成，即 $\{1FXq, ec20, 8f74\}$。

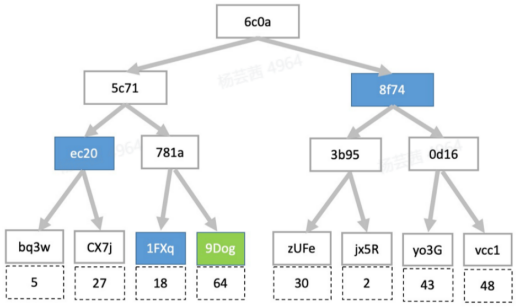

图 2.1 Merkle 树示例图

逐层迭代计算，计算目标数据 64 的哈希值 9Dog，并与其左兄弟节点 1FXq 级联哈希，得到父节点 781a。将计算得到的 781a 与其左兄弟节点 ec20 级联哈希，得到更高一级的父节点 5c71。将 5c71 与其右兄弟节点 8f74 级联哈希，最终计算出该树的根哈希值。验证者将计算得出的根哈希值与预存的区块头根哈希 6c0a 进行比对。若两者完全一致，则证明数据块 64 确实存在于该数据集中且未被篡改。通过此过程，轻节点无需下载完整的交易数据集，仅需请求 $O(\log n)$ 长度的证明即可确认交易的确认状态，极大地降低了分布式环境下的通信开销。

2.4　本章小结

本章系统梳理了可验证查询方案所依赖的基础密码学技术，包括哈希函数、伪随机函数、密码学累加器与 Merkle 树等。上述工具为后续方案的安全性、完整性与高效性提供了理论与实现基础。理解这些原理，有助于深入把握后文各模块的设计思路与安全分析。

第三章　方案模型

本章将系统性介绍本文提出的可验证区块链查询方案的整体架构、数据与查询建模、核心算法流程及安全性目标。通过形式化建模与安全分析，为后续方案实现与性能评估奠定理论基础。

3.1　系统模型

本文提出了一种新的 vscc 框架，研究了在密态区块链数据库上的可验证查询处理。如图**??**展示了本文的系统模型。DO 负责系统初始化、数据加密、索引构建与更新，并将密文索引上传至区块链，而真实数据以加密形式存储在链下数据库中。DU 作为轻量级节点，在本地利用授权密钥材料生成查询陷门，并向 SP 发起查询请求。SP 基于链上索引执行检索，生成结果集及验证对象后广播给其他全节点进行共识投票。投票结果同时记录在区块链上，以支持结果审计和抗抵赖。最终，SP 将带有共识签名的查询结果返回给 DU，DU 在本地完成验证与解密。

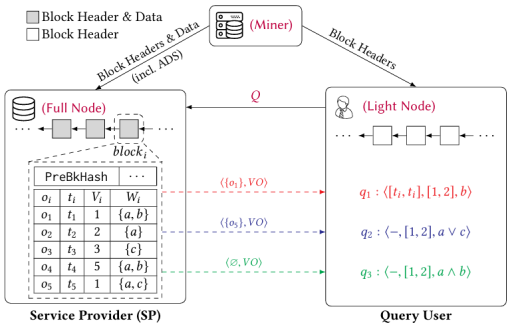

Figure 3: System Model of vChain

图 3.1　vChain 系统模型

为了实现对存储在区块链或分布式数据库中复杂数据的统一处理，我们对数据对象及用户查询请求进行形式化建模。

我们将系统中的数据对象建模为一系列序列化对象 $o_1, o_2, \ldots, o_n$，每个数据对象 o_i 都可以表示为三元组:

$$o_i = \langle T_i, V_i, W_i \rangle \tag{3-1}$$

其中，T_i 表示对象的时间属性，在本文场景下可等同理解为对象写入时对应的区块高度；V_i 表示对象的数值属性向量，为了便于说明，下文主要以一维数值属

性为例；W_i 表示对象的关键字集合。例如，考虑一条供应链金融中的资产凭证记录 $o_i = \langle$2024-03-22; 10：00：00, ; 5000, ;“Alice”,“Bob”$\rangle$。该对象包括一个交易时间戳 T_i = 2024-03-22; 10：00：00，属性值集合 V_i 记录了该笔资产的金额 5000 元，而 W_i 则包含了该资产的发行方“Alice”、接收方“Bob”。

基于上述建模，用户发起的复杂检索请求可以表示为一个三元组查询：

$$Q = \langle [t_s, t_e], \Phi_W, \Phi_V \rangle \tag{3-2}$$

其中，$[t_s, t_e]$ 是查询时间区间，Φ_W 为定义在关键字上的布尔表达式，仅支持合取与析取，Φ_V 为定义在数值属性上的范围查询。当查询不包含某类条件时，可将对应谓词记为空。例如，一个典型的审计查询可以表示为：$Q = \langle$[2024-01, 2024-03], ; [1000, 5000], ; (Buyer: Al Status: Approved)$\rangle$。该查询将搜索所有时间戳在 2024 年第一季度内、金额位于 [1000, 5000] 范围内，且属性同时满足买方为 Alice 和状态为“已审核”的交易记录。

由于系统采用滑动窗口模型，每个区块仅承诺以该块为右端点的当前窗口状态，因此针对时间区间 $[t_s, t_e]$ 的查询，需要被划分为一组与时间窗口状态对齐的子查询，每个子查询都对应某个窗口末端区块上的索引根，从而保证查询、验证与链上状态严格对应。

3.2 系统形式化定义

本文提出的可验证查询方案由以下六个多项式时间算法组成：

（1） $\mathsf{Setup}(1^\lambda) \rightarrow (msk, params)$：由数据拥有者（DO）执行。输入安全参数 λ，输出系统主密钥 msk（包含对称密钥 mk 和累加器私钥 sk）及公开参数 $params$（包含累加器公钥 pk 和全局注册表配置）。

（2） $\mathsf{GenIndex}(D, msk) \rightarrow (I, acc)$：由 DO 执行。输入原始数据集 D，利用 msk 对数据进行预处理、规范化及加密映射，生成密文索引 I（即 `IndexedObject`）和对应的累加器摘要 acc。

（3） $\mathsf{Trapdoor}(Q, msk) \rightarrow T_Q$：由数据用户（DU）执行。输入查询请求 $Q = \langle [t_s, t_e], [a, b], \phi \rangle$，利用授权密钥材料生成包含密文令牌（Tokens）的查询陷门 T_Q 。

（4） $\mathsf{Query}(I, T_Q, params) \rightarrow (R, \pi)$：由服务提供商（SP）执行。SP 在密文索引 I 上检索满足陷门 T_Q 条件的结果集 R，并根据集合运算逻辑生成对应的密码学

证明 π。

（5） $\mathsf{Verify}(R, \pi, params, acc) \rightarrow \{0, 1\}$：由 DU 或全节点执行。利用公开参数和累加器摘要验证结果集 R 的可靠性与完整性。若验证通过则输出 1，否则输出 0。

（6） $\mathsf{Decrypt}(R, msk) \rightarrow D_R$：由 DU 执行。对通过验证的结果集进行解密，恢复出原始明文数据。

表 3.1　方案主要符号及其含义

符号	含义说明
λ	系统安全参数
mk	对称主密钥 (Master Key)
sk, pk	累加器私钥与公钥
O_i	第 i 个原始数据对象
IO_i	密文索引对象 (Indexed Object)
$acc(X)$	集合 X 的累加器摘要值
$Token$	加密令牌 (Token)
ID_{kw}, ID_{pf}	注册表编号
$P_X(z)$	特征多项式
T_Q	查询陷门 (Trapdoor)
π_{opt}	集合运算证明 (Proof)
$e(\cdot, \cdot)$	双线性配对运算

3.3　威胁模型与安全目标

3.3.1　安全模型

本方案的安全性通过攻击者 $\mathscr{A}$ 与挑战者 $\mathscr{B}$ 之间的形式化博弈进行定义。

3.3.1.1　关键字猜测攻击安全博弈

由于本方案中的关键字令牌 $Token_{kw}$ 存储在公开的注册表中，必须证明即使敌手获知部分令牌和对应的索引 ID，也无法在离线状态下通过枚举关键字来识别未知的陷门。

初始化 (Setup)：挑战者 $\mathscr{B}$ 运行密钥生成算法，生成主密钥 mk 和累加器参数

(pk, sk)。$\mathscr{B}$ 将安全参数 λ 和公钥 pk 发送给敌手 $\mathscr{A}$。

陷门询问 (Trapdoor Query)：$\mathscr{A}$ 适应性地选择关键词集合 W 提交给 $\mathscr{B}$。$\mathscr{B}$ 返回由 mk 加密生成的令牌集合 $T_W = \{F(mk, kw) \mid kw \in W\}$。

挑战 (Challenge)：$\mathscr{A}$ 选择两个从未被询问过且等长的目标关键词集 W_0^* 和 W_1^* 提交。$\mathscr{B}$ 随机选择 $b \in \{0, 1\}$，计算挑战索引 I_b^* 并发送给 $\mathscr{A}$。

猜测 (Guess)：$\mathscr{A}$ 输出猜测值 $b' \in \{0, 1\}$。如果 $b' = b$，则称 $\mathscr{A}$ 赢得博弈。

安全定义：若对于任何多项式时间敌手 $\mathscr{A}$，其赢得游戏的优势 $\mathrm{Adv}_{\mathscr{A}}^{\mathrm{KGA}}(\lambda) = |\Pr[b' = b] - 1/2|$ 是可忽略的，则称方案满足抗关键词猜测攻击安全性。

3.3.1.2 密文不可区分性

为了证明非对称累加器部分的安全性，我们参考经典的 IND-CKA (Chosen-Keyword Attack) 模型。该模型确保了累加器摘要 $AccValue$ 不会泄露其包含的特定特征 ID 的任何代数信息

Setup：$\mathscr{B}$ 生成累加器公私钥对 (pk, sk) 并将 pk 发送给 $\mathscr{A}$。

Interrogation：$\mathscr{A}$ 提交多个集合 S_i，$\mathscr{B}$ 返回对应的累加器摘要 $acc(S_i) = g^{P_{S_i}(s)}$。

Challenge：$\mathscr{A}$ 选取两个不同但基数相同的集合 S_0^*, S_1^*。$\mathscr{B}$ 随机选取 $b \in \{0, 1\}$ 并返回挑战摘要 $acc(S_b^*)$。

Guess：$\mathscr{A}$ 输出猜测 b'。

安全定义：该安全性归约至双线性群上的 q-强 Diffie-Hellman (q-SDH) 困难性假设。在 q-SDH 假设成立的前提下，$\mathscr{A}$ 无法以不可忽略的优势区分两个不同集合的累加结果。

3.3.2 安全目标

在本方案中中的分布式架构中，信任边界仅存在于数据拥有者（DO）与本地存储之间，外部实体均被视为潜在的对手。SP 负责托管密文索引并执行检索计算。由于程序故障、恶意攻击或商业利益驱动（如减少算力开销），SP 被认为是非诚实的。它可能修改存储的对象内容或伪造不存在的对象，也可能在复杂的集合运算（交、并、差）中遗漏部分符合条件的子集。在分布式交易或审计场景中，DU 可能会发起抵赖攻击（Repudiation Attack）。当 DU 接收到正确的查询结果后，为了逃避服务付费或恶意诬陷 SP，DU 可能会否认收到过正确的结果，声称 SP 提供的证

明 π_{opt} 无效或数据不完整。针对上述复杂的威胁环境，本方案旨在实现以下详尽的安全目标：

（1） **完整性（Integrity）：**确保 SP 无法篡改存储的对象内容或伪造不存在的对象。任何对存储对象的修改都必须被 DU 发现。

（2） **正确性（Correctness）：**确保 SP 在执行集合运算（交、并、差）时，返回的结果集必须完全包含所有满足查询条件的对象，且不包含任何不满足条件的对象。

（3） **抗抵赖性（Non-repudiation）：**确保 DU 无法否认收到过正确的查询结果。任何 DU 的抵赖行为都应被系统记录并可供审计。

3.3.3 本章小结

这里 AI 率很高本章围绕可验证区块链查询方案，系统阐述了系统模型、数据与查询建模、核心算法流程及安全性目标。通过形式化定义与安全博弈分析，明确了各参与方的职责与潜在威胁，并提出了完整性、正确性和抗抵赖等关键安全目标。这些内容为后续方案实现、优化与性能评估提供了理论基础和设计依据。

第四章　初始化

系统初始化是整个外包检索协议的基石。在数据上传前，DO 必须对明文空间进行代数化映射，构建从高维文本与连续数值到离散有限域元素的映射字典。该过程涵盖密码学参数生成、关键字词表初始化以及数值前缀空间初始化。

4.1　密码学参数生成

下表**??**展示系统初始化阶段所需参数：

表 4.1　初始化参数及其含义

符号	含义说明
λ	系统安全参数
w	滑动窗口大小
f	固定扇出
$maxID$	对象注册空间上届
l_v	数值编码位长

其中，$maxID$ 用于约束累加器绑定的对象全集规模，l_v 用于确定前缀树高度。

4.2　词表初始化

我们期望将关键字密文压缩到 NonZeroU16 有两个原因。一是 AttDigest 索引节点会存储加密后的关键字集，相比直接存储通常是 128 位经过 PRF 加密后的数据，使用某种方式将密文限制在 NonZeroU16 的集合元素更节省空间。二是对象关键字集是否包含请求关键字时，我们需要依靠累加器给出计算结果和证明。由于输入是任意字符串集合，若仅简单采用字节阶段压缩，从数学上无法避免碰撞。并且从效率上考虑，若关键字总量很小，其映射结果稀疏，还必须设置构建公私钥的必要参数全集大小为 65535。由于用户知道关键字全集，我们考虑让用户离线构建一个的键值对表上传至区块链，具体过程如下：

对于系统关键字全集中的每一个明文关键字 w, 用户先在做规范化处理，随后计算一个稳定的搜索标签 $tok(w) = PRF_{kw}(w)$ 并排序，将排序后的第 i 个关键字

分配编号 i+1, 这样就会得到一个在固定全集上的伪随机编号。

$$R : tok(w) \leftrightarrow kw_{id} \tag{4-1}$$

此表构建完成后，用户将词表作为系统初始化参数的一部分发布到链上。查询用户使用的标签会现在词表中校验是否存在，非空才会在链上执行查询过程。

但这样有一个缺陷在于，该设计没有限制住关键字对应的 ID 大小，词表可以实现零冲突但理论上可以无限增长，超出累加器所能处理的范围。

4.3 前缀表初始化

通过前缀转换函数[xu2019vchain]$\mathbb{TR}(\cdot)$，一个数值可以转换为一组二进制元素。例如，4 的二进制表示为 100，它的前缀集可以表示 $\mathbb{TR}(4) = \{1*, 10*, 100\}$, 其中 $*$ 表示通配符匹配操作符。我们通常取能表达整个数据集的位数作为前缀二叉树深度，对于单维范围 $[\alpha, \beta]$, 我们首先将 α, β 表示为二进制格式，将其视为整个二进制空间集树的最小叶节点，所能精确覆盖整个范围 $[\alpha, \beta]$ 的最小树节点集合就是所找的前缀集合。例如，对于一维范围 $[0, 6]$，首先构建位长为 3 的二进制空间集树，如图**??**先将 0 和 6 作为树的两个叶子节点，最后找到覆盖其整个范围 $[0, 6]$ 的树节点的最小集合 $\{*, 10*, 100\}$。通过上述变换，查询一个数值 v 是否在区间 $[\alpha, \beta]$

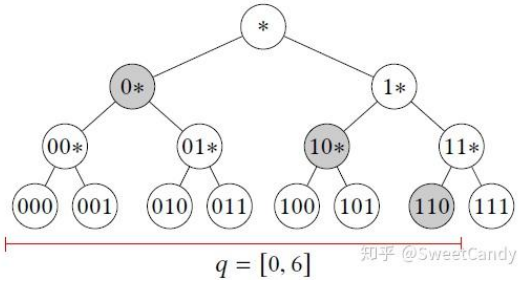

图 4.1 二叉前缀树

内，就变成了将 v 的变换前缀集合与 $[\alpha, \beta]$ 的等价集合进行布尔查询。在上述例子中，$4 \in [0, 6]$ 因为 $\{1*, 10*, 100\} \cap \{0*, 10*, 110\} = \{10*\} \neq \varnothing$, $7 \notin [0, 6]$ 因为 $\{1*, 11*, 111\} \cap \{0*, 10*, 110\} = \varnothing$。

利用前缀转换，数值加密后的的范围查询实际上等同于在密文关键字查询，也就需要构造前缀集合的密文词表。用户已知表达整个数据集的所需位长，预先枚举所有前缀，每个前缀经过 PRF 得到 token，token 再映射到词表 ID。

这样做有两个缺陷，一是前缀表大小是所表达数值个数的两倍。若数据表示位数 4，则枚举前缀集合共有 31 个元素。我们限定集合元素大小为 U16, 那么数据最大位长为 15，所能表示最大数据为 32768。二是若用户给定的位数参数很小，则候选空间本身就很小，结合查询模式、访问模式、频率信息，服务端很容易做枚举或统计推断。第二个问题没有在词表中出现是因为词表的输入空间是无限的，而数值的输入空间有限。不过这点待定，如果服务器不知道当前索引是范围索引呢? 在它视图下仅仅是一个特殊的关键字索引?

4.4　本章小结

本章详细介绍了系统初始化阶段的关键步骤，包括密码学参数生成、密文词表初始化以及前缀表初始化。通过规范化、确定性令牌生成和保序扰乱等技术手段，系统成功将高维文本与连续数值映射为离散的有限域元素，为后续的加密索引构建和可验证查询处理奠定了坚实的基础。

第五章　索引构建

本文从整体上采用滑动窗口机制，也就是索引会有一个窗口生命周期管理，在生成一个新区块时，旧块数据会在索引中删除，新块数据会加入索引。索引始终维护的是最近窗口大小的数据状态，链尾区块记录的也就是当前索引的最新快照。

5.1　密文索引构建

5.1.1　对象注册

本文采用的密码学累加器对输入全集大小敏感，其证明生成复杂度为 $O(N_1 \cdot N_2)$，其中 N_1, N_2 分别为输入集 X_1, X_2 的大小。相较于其他累加器[xu2019vchain]，该方案虽支持交、并、差等嵌套集合操作，但需以更大的证明为代价，且公钥大小与输入集合元素的全集大小呈二次关系。若直接将滑动窗口中的数据对象通过加密散列编码为 256 位整数，公钥规模将达 $(2^{256})^2 = 2^{512}$，难以接受。

为此，我们提出对象注册索引 ObjReg，为最近 $2^K - 1$ 个区块内的数据对象分配有限 ID，并为后续属性与范围索引提供统一标识。通过设定最大 ID（记为 $MaxID$），将参与累加器运算的输入集大小限制在 $MaxID$ 内，从而控制公钥规模。例如，在实验中将 $MaxID$ 设为 2^{12}，公钥大小即为 $(MaxID)^2 = 2^{24}$。该机制同时保证任意连续 $2^K - 1$ 个区块内的数据对象均具有唯一 ID，而本文涉及的集合操作仅在这一窗口范围内进行，因此所有相关对象均可确保 ID 不重复。

ObjReg 是一棵扇出为 f 的完全平衡 Merkle 树，其根哈希写入区块头，用于承诺以该区块结束的窗口状态，同时计入区块体以供一致性检查。对象注册时，矿工为其分配一个随计数器递增并模 $MaxID$ 的 ID。由于树结构固定，ID_i 可视为以 f 为基数的整数：

$$ID_i = \sum_{j=0}^{h-1} d_j \cdot f^j \quad \text{其中} d_j \in [0, f-1] \tag{5-1}$$

每位 d_j 对应第 j 层的子节点选择，Merkle 树子节点从 1 编号时，路径取 $d_j + 1$。如图 **??** 所示，设扇出 $f = 3$，对象 o_6 的 ID 为 6，其三进制表示为 020，对应树路径为 $(1, 3, 1)$。

叶节点存储对应对象标识符位置的加密承诺及其哈希，内部节点仅存储子节

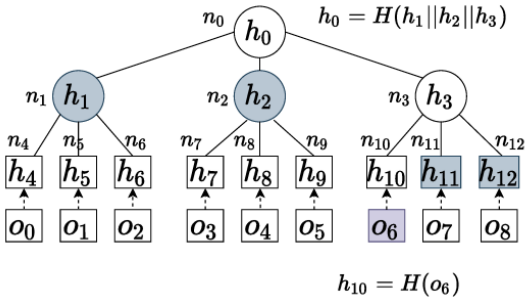

Fig. 6: Object Registration Index

图 5.1 ObjReg 树结构

点哈希的聚合值。每个区块维护自身对应的 ObjReg 根，系统中不存在一个横跨全链的单一全局根，而是形成一条随区块演化的状态序列。后续索引不再引用原始对象，而是统一引用这些经过 ObjReg 分配的对象标识符。

5.2 关键字索引 AttDigest

起初我们设想 AttDigest 继续按照 ObjReg 的 ID 位置插入，那么其本质就是带属性摘要的 ObjReg 镜像树。这样更新简单，但剪枝效果很弱，因为树的空间邻近性来自 ID，而不是来自属性相似性，所以我们需要找到某种组织方式使得 AttDigest 按相似语义聚集。回忆先前设计，系统初始化时就已经把关键词压缩成了稳定稠密的词表编号，所以关键字经过 PRF 在词表中对应的 kw_{id} 就是此时 AttDigest 的索引键。语义上搜索标签 token 等于词表编号 kw_{id}。自然而然地，关键字索引 AttDigest 设计为和 ObjReg 一样的固定扇出、按 kw_{id} 唯一路径定位的认证倒排树。每个叶子唯一对应一个 kw_{id}，因此 AttDigest 的路径由 kw_{id} 唯一确定，而不再依赖对象分布或插入顺序。下面给出每个索引节点定义：

定义 1. *(AttDigest Leaf Node)* 设叶节点 n 对应关键字 k，其字段定义如下：

- kw_{id} = 当前叶对应的关键词编号
- S_n = k 对应的对象 ID 集合
- $acc_n = acc(S_n)$
- $cnt_n = |S_n|$
- $h_n = H(kw_{id} \parallel acc_n \parallel cnt_n)$

定义 2. *(AttDigest Non-Leaf Node)* 定义非叶节点的子节点为 $c_1, c_2, \ldots, c_F$，其字段定义如下：

- $acc_n = acc(K_n)$，其中 K_n 是当前子树下非空关键字编号集合
- $h_n = H(acc_n \parallel h_{c_1} \parallel h_{c_2} \parallel \cdots \parallel h_{c_F})$（如果 n 不是根节点）
- $h_n = H(acc_n \parallel h_{c_1} \parallel h_{c_2} \parallel \cdots \parallel h_{c_F} \parallel height)$（如果 n 是根节点）

对于叶节点 n，若当前窗口内没有任何对象包含关键字 k, 则令 $S_k = \emptyset$, 并计算相应空集合累加器摘要与空叶哈希。请注意叶节点存储的是 kw_id 与对象标识符集合的认证摘要，而不是明文关键字或者 32 字节搜索标签本身。这样既压缩了节点存储开销，又避免了任意字符串域直接映射到小整数域所带来的碰撞问题，因为系统中的整数编号不是由简单压缩得到，而是由共享词表对全体关键字进行一致编号后产生。

对于内部节点 n，实际上其所覆盖的关键字编号区间 K_n 是由树结构隐式决定的，但我们仍存储其累加器摘要 acc_n 是为了批量查询优化与空子树快速判断，它并不是单关键字查询正确性的必要条件。单关键字查询的正确性已经可以由 kw_{id} 唯一路径定位的叶子认证路径保证；该字段的存在只是为了在将来支持多关键字联合调度、批零空子树等工程优化时，提供附加的认证辅助。

如算法表示：在窗口末端 b_h 处，系统首先枚举该窗口内的所有对象标识符。对于每个对象，获取其对应的关键字编号集合，并将该对象的标识符分别添加到每个关键字所对应的倒排集合中。全部对象处理完毕后，即可得到当前窗口下完整的倒排集合族。随后，对每个关键字的倒排集合计算累加器摘要，形成树的叶节点，再自底向上递归计算内部节点的累加器摘要与哈希值，最终得到当前窗口的 attdigest 根哈希。该根哈希被写入区块头，作为窗口 b_h 对关键字索引状态的唯一承诺。

举例：

5.3　密文索引更新

由于系统采用滑动窗口模型，当新区块 b_{h+1} 到达时，索引更新不是简单插入，而是一个先删除过期对象，再插入新对象的状态转移过程。

设当前窗口对应区间为 $[h-w+1, h]$。当新区块 b_{h+1} 到达时，若窗口已满，则原窗口左端块 b_{h-w+1} 中的对象全部过期，需要从 ObjReg、AttDigest 和 RangeDigest 中删除; 随后，再将新区块中的对象注册进 ObjReg，并加入两类索引。这样，系统状态从窗口 $[h-w+1, h]$ 平滑转移到窗口 $[h-w+2, h+1]$。

5.3.1 ObjReg 更新

对每个过期对象，系统依据其旧的对象标识符，将对应叶节点恢复为空值，并沿路径认证自底向上重算到根; 对于每个新对象，系统分配新的对象标识符或重复利用已释放的标识符，并填充到相应叶位置，再重算到根，从而得到新的 ObjReg 根。

5.3.2 AttDigest 更新

系统为高效维护关键字倒排索引，额外维护了一个反向映射，记录每个对象标识符 id_{o_i} 所对应的全部关键字编号。这样，当对象过期时，系统无需重新解析其明文字段，可直接通过该映射定位到受影响的叶节点。

对于每个过期对象，系统首先获取其关联的关键字编号集合。随后，对集合中的每个关键字，从对应的倒排集合中移除该对象的标识符，并重新计算受影响叶节点的累加器摘要与计数信息，再沿关键字路径自底向上更新祖先节点的摘要与哈希值。对于新区块中的新对象，则执行相反流程：读取其关键字编号集合，将对象标识符依次插入相应倒排集合，并沿路径完成更新。

因此，AttDigest 的更新代价不取决于整棵树的大小，而只取决于过期与新增对象的关键字数量以及树高。由于节点键由 kw_id 唯一确定，更新过程中不会发生树结构重排或者节点语义重写，从而保证该索引在滑动窗口模型下是可删除且可增量维护的。

举例：

第六章　密文查询与验证

6.1　单关键字查询

查询阶段，设用户在某个窗口状态 h 下查询关键字 w。用户首先在本地执行与初始化阶段相同的关键字规范化和带密钥映射，得到 $tok(w) = PRF_{kw}(w)$, 然后在共享词表中查得其关键字编号 $kw_{id} = \mathscr{R}(tok(w))$。

若词表中不存在该关键字编号，则说明该关键字不再系统词汇表中，查询结果直接为空。若存在对应编号 k, 则 SP 在 AttDigest 中按照 k 的唯一路径定位到对应叶节点，并返回当前窗口下该关键字对应的结果对象标识符集合 R_k 以及该叶节点到 AttDigest 根的 Merkle 认证路径 π_k。于是，查询结果可表示为 $\langle R_k, \pi_k \rangle$。

与[chen2019mechanics]的差异在于，系统不再需要从根开始遍历大量对象子树，而不需要对当前节点关键字集合与查询关键字是否相交反复给出交空证明。因为在新的 AttDigest 中，每个关键字编号对应一个唯一叶子位置，查询只需定位到该叶并返回结果集合以及认证路径即可。

验证阶段，用户收到 $\langle R_k, \pi_k \rangle$ 后在本地先计算集合大小 $cnt_k = |R_k|$, 再利用私钥对集合 R_k 重算累加器摘要 acc_k 进而重算叶节点哈希 $h_k = H(k||acc_k||cnt_k)$, 最后结合路径 π_k 自底向上重算到根 $root'_k$。若 $root'_k$ 与链上公布的 AttDigest 根比较。若二者一致，则说明 SP 返回的结果集合与链上状态一致，用户接受该结果否则拒绝。

若 SP 漏返回对象、多返回无关对象、返回空集欺骗用户没有结果、返回其它关键字的结果甚至伪造 Merkle 路径，都会因为根对比失败而被发现。

6.2　布尔查询

设关键字布尔表达式为 $\Phi_w(w_1, w_2, \ldots, w_m)$, 其中仅仅包含合取与析取。对于表达式中的每个原子关键字 w_j, 用户现在本地得到对应关键字编号 k_j, 再要求 SP 分别返回叶节点倒排结合及其认证证明 $\langle R_{k_j}, \pi_j \rangle$。接着 SP 会根据布尔表达式 Φ_w 对这些集合执行交、并等集合运算。例如，当 $\Phi_W = w_1 \wedge (w_2 \vee w_3)$ 时，SP 先利用并集证明得到 $R_{23} = R_{k_2} \cup R_{k_3}$, 再利用交集证明得到 $R_W = R_{k_1} \cap R_{23}$。整个过程

中，SP 不仅返回最终结果集合 R_W，还需返回每一步集合运算相对应的累加器证明 π_{op}，使得用户能够验证布尔结果确实由已认证的各关键字倒排集合按给定布尔逻辑组合而成。

这种设计下，SP 的恶意行为可能出现在两个层面。第一，SP 可能会在某个原子关键字查询结果中遗漏对象、添加无关对象或返回错误路径，但这类攻击会在原子结果的单关键字验证阶段被发现。第二，SP 可能会在集合运算过程中作恶，例如再并集结果中遗漏应出现的元素、加入不属于任一集合的元素、或在交集结果中省略真实公共元素、加入仅属于某一侧属于集合的元素。由于每一步中间结果都必须通过相应的累加器运算证明验证，因此上述伪造行为均会导致证明不成立，用户据此拒绝该查询结果。

因此，布尔查询的验证结果可以概括为: 先认证每个原子关键字对应的结果集合，再逐步认证每一步集合运算，最终得到与表达式 Φ_w 语义一致的可验证结果集合 R_w。

6.3 范围查询

6.4 聚合验证

第七章　总结与展望

致　谢

参考文献

[1] KIM T H, PARK J H. Sustainability and Animal Power in Modern Construction[J]. Journal of Sustainable Construction, 2021, 12(1): 1-12.

[2] CHEN L, ZHANG W. Experimental Analysis of Animal Power in Structural Stability[J]. Journal of Structural Mechanics, 2023, 45(2): 88-97.

[3] BROWN M, SMITH J. Case Studies on the Efficiency of Cattle and Horse Power in Contemporary Construction[J]. Journal of Civil Engineering Case Studies, 2022, 8(4): 345-356.